



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Ingeniería en Software

PRÁCTICA EXPERIMENTAL VII

**Implementación y Consumo de Servicios Web
Escalables (REST y SOAP)**

Materia: Aplicaciones Web ‘A’

Docente: Gleiston Guerrero Ulloa

Período Académico: SPA 2025-2026

Estudiante: Eduardo Reinoso Vélez

Repositorio de la actividad:

<https://github.com/ereinosov/practicaUnidadVII>

Quevedo - Ecuador
Febrero - 2026

Investigación bibliográfica: Servicios web, arquitecturas REST y SOAP, e integración de APIs externas en sistemas distribuidos

El desarrollo de aplicaciones empresariales modernas se sustenta en la capacidad de los sistemas de software para comunicarse entre sí de forma eficiente, independientemente del lenguaje de programación, la plataforma tecnológica o la ubicación geográfica de los componentes involucrados. Esta necesidad ha propiciado la consolidación de los *servicios web* como un paradigma esencial dentro de la arquitectura distribuida contemporánea, al ofrecer mecanismos estandarizados para la integración de sistemas heterogéneos [1, 2].

Los servicios web permiten la interoperabilidad entre aplicaciones mediante el uso de estándares abiertos basados en protocolos de red ampliamente adoptados. Dentro de este contexto, dos enfoques han adquirido especial relevancia: la arquitectura *REST* (Representational State Transfer) y el protocolo *SOAP* (Simple Object Access Protocol), cada uno con principios de diseño, ventajas técnicas y ámbitos de aplicación claramente diferenciados [3, 4].

El presente documento constituye un marco teórico de carácter estrictamente académico, orientado a contextualizar el estudio de la implementación y el consumo de servicios web escalables. Se abordan de manera sistemática los fundamentos de los sistemas distribuidos, los protocolos de comunicación, los formatos de intercambio de datos, las arquitecturas de servicios, los mecanismos de autenticación, la computación en la nube y los principios de seguridad y escalabilidad. Cada uno de estos temas se desarrolla con referencia a literatura científica relevante, con el objetivo de proporcionar una base conceptual sólida que facilite el análisis crítico de las tecnologías estudiadas.

1. Servicios web

Esta sección introduce el concepto de servicio web y su evolución, destacando las propiedades que permiten integrar aplicaciones heterogéneas y sostener arquitecturas distribuidas modernas. El objetivo es establecer una base conceptual que sirva de referencia para comparar enfoques y comprender decisiones de diseño posteriores.

1.1. Definición y características fundamentales

Un servicio web es un sistema de software diseñado para soportar la interacción interoperable entre máquinas a través de una red, mediante la exposición de funcionalidades accesibles a través de interfaces estándar descritas en un formato procesable por máquinas [2]. Esta concepción enfatiza el papel de los servicios web como mecanismos de abstracción que desacoplan la lógica interna del proveedor de la lógica del consumidor, habilitando la integración de sistemas heterogéneos sin imponer restricciones sobre la tecnología subyacente de cada componente.

Las características que distinguen a los servicios web de otros mecanismos de integración incluyen la *interoperabilidad*, entendida como la capacidad de comunicación entre sistemas desarrollados con tecnologías dispares; la *independencia de plataforma*, que posibilita el intercambio de mensajes sin depender del sistema operativo o lenguaje de programación; y la *orientación a estándares abiertos*, que garantiza neutralidad tecnológica y fomenta la adopción generalizada [1, 2].

1.2. Evolución y relevancia actual

La evolución de los servicios web se ha desarrollado desde las arquitecturas orientadas a servicios (SOA) predominantes a finales de la década de 1990, basadas principalmente en SOAP y WSDL, hacia enfoques más ligeros y flexibles como REST, propuesto formalmente en la disertación doctoral de Roy Fielding en el año 2000 [3]. Posteriormente, la proliferación de dispositivos móviles, la computación en la nube y el surgimiento del paradigma de los microservicios han reforzado el papel de los servicios web como unidades fundamentales de composición de sistemas distribuidos [5, 6].

En la actualidad, los servicios web constituyen la base operativa de la mayoría de las plataformas digitales a gran escala, desde redes sociales hasta sistemas financieros y de salud pública, lo que los convierte en un componente crítico de la infraestructura tecnológica global. La transición desde arquitecturas monolíticas hacia composiciones de microservicios independientes ha intensificado la dependencia de las organizaciones en interfaces bien definidas y contratos estables de comunicación [1, 5].

2. Sistemas distribuidos

En esta sección se describen los fundamentos de los sistemas distribuidos y sus modelos de arquitectura, con énfasis en los retos asociados a la comunicación por red y la coordinación entre componentes. Estos conceptos resultan esenciales para comprender por qué los servicios web requieren decisiones explícitas sobre consistencia, tolerancia a fallos y patrones de resiliencia.

2.1. Concepto y modelos de arquitectura

Un sistema distribuido es un conjunto de procesos de cómputo autónomos, ubicados en diferentes nodos de una red, que cooperan para alcanzar un objetivo común y que, desde la perspectiva del usuario, se presentan como un sistema único y coherente [7]. Esta definición implica que los componentes no comparten memoria ni reloj global, y que la coordinación entre ellos se realiza exclusivamente mediante el intercambio de mensajes, lo que introduce una clase de problemas cualitativamente distintos a los de los sistemas centralizados.

Entre los modelos arquitectónicos más relevantes se encuentran el modelo *cliente-servidor*, en el cual los clientes solicitan servicios a servidores especializados; el modelo *peer-to-peer*, donde los nodos actúan simultáneamente como proveedores y consumidores; y el modelo *multi-tier*, que separa la lógica de presentación, negocio y datos en capas independientes [1, 7].

2.2. Desafíos en sistemas distribuidos

La construcción de sistemas distribuidos introduce desafíos que no se presentan en sistemas monolíticos. El *teorema CAP* establece que no es posible garantizar simultáneamente consistencia, disponibilidad y tolerancia a particiones de red, por lo que todo sistema distribuido debe priorizar y gestionar compromisos entre estos atributos [7]. A esto se suman problemas como la latencia de red, los fallos parciales, la heterogeneidad de los nodos y la complejidad de las transacciones distribuidas, que requieren soluciones específicas a nivel de diseño arquitectónico.

Para mitigar estos desafíos se emplean algoritmos de consenso como Paxos o Raft, mecanismos de replicación y patrones de diseño como *circuit breaker* y *compensación transaccional*, ampliamente documentados en la literatura especializada [8, 9].

3. Protocolos de comunicación en la web: HTTP y HTTPS

Esta sección examina los fundamentos del protocolo HTTP y su variante segura HTTPS, que constituyen la base de transporte sobre la que operan los servicios web RESTful y, con frecuencia, los servicios SOAP. La comprensión de su semántica, versiones y modelo de seguridad resulta indispensable para el diseño correcto de APIs y para la evaluación de sus propiedades de rendimiento y protección.

3.1. El protocolo HTTP y su evolución

El *Hypertext Transfer Protocol* (HTTP) es el protocolo de nivel de aplicación que rige el intercambio de mensajes en la Web. Su diseño sigue un modelo solicitud-respuesta sin estado (*stateless*), en el que cada transacción es independiente y no conserva contexto entre peticiones sucesivas [10]. Esta característica implica que toda la información necesaria para procesar una solicitud debe estar contenida en el propio mensaje, lo que facilita la escalabilidad horizontal de los servidores pero exige mecanismos adicionales para la gestión del estado de las sesiones de usuario.

La versión HTTP/1.1 introdujo las conexiones persistentes (*keep-alive*) que reducen la latencia asociada al establecimiento repetido de conexiones TCP. HTTP/2 incorporó multiplexación de flujos, compresión de cabeceras mediante HPACK y priorización de solicitudes, lo que mejoró sustancialmente el rendimiento en aplicaciones modernas con múltiples recursos por página. HTTP/3, la versión más reciente, abandona TCP en favor del protocolo QUIC sobre UDP, eliminando el problema de bloqueo de cabecera de línea e integrando el establecimiento de conexiones seguras en un único viaje de red [10]. Los métodos principales de HTTP —GET, POST, PUT, PATCH, DELETE, OPTIONS y HEAD— definen la semántica de las operaciones sobre los recursos y su correcta utilización es un requisito fundamental para el diseño de APIs RESTful conformes con las restricciones del estilo arquitectónico de Fielding [3, 11].

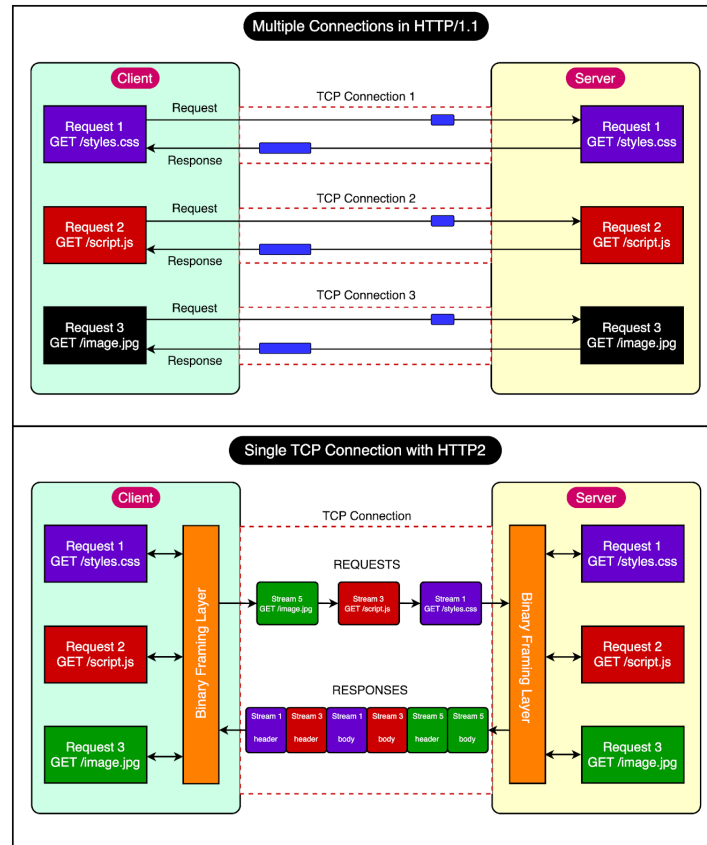


Figura 1: Evolución de las versiones del protocolo HTTP mostrando las mejoras en multiplexación y rendimiento.

3.2. HTTPS y seguridad en la capa de transporte

HTTPS es la variante segura de HTTP obtenida mediante la combinación con el protocolo *Transport Layer Security* (TLS), el cual proporciona confidencialidad mediante cifrado simétrico, integridad mediante códigos de autenticación de mensajes y autenticación del servidor mediante certificados X.509 [12]. TLS 1.3, estandarizado en 2018, redujo el número de intercambios necesarios para establecer una conexión segura a un único viaje de ida y vuelta, eliminando algoritmos criptográficos obsoletos y mejorando el rendimiento sin comprometer la seguridad.

La adopción masiva de HTTPS, impulsada por iniciativas como *Let's Encrypt* y las políticas de los navegadores que marcan los sitios HTTP como inseguros, ha convertido a este protocolo en un requisito de facto para cualquier servicio web expuesto públicamente [12]. La correcta configuración de HTTPS implica además la habilitación de *HSTS* (HTTP Strict Transport Security) para prevenir ataques de degradación de protocolo y la implementación de *Certificate Transparency* para detectar certificados emitidos de forma fraudulenta.

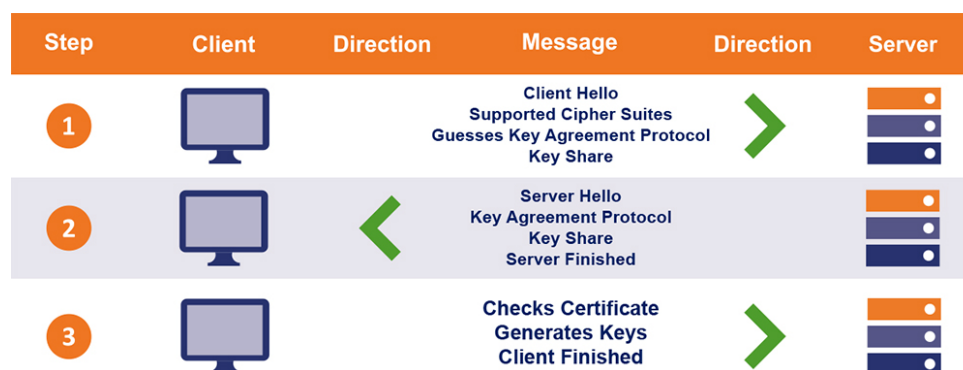


Figura 2: Diagrama del proceso de negociación TLS 1.3, mostrando el intercambio ClientHello, ServerHello y establecimiento de conexión segura.

4. Formatos de intercambio de datos: XML y JSON

Esta sección analiza los dos formatos predominantes para el intercambio de datos en servicios web. La elección entre ellos no es únicamente técnica, sino que tiene implicaciones directas en la legibilidad, el rendimiento, la capacidad de validación y la interoperabilidad del sistema.

4.1. XML: Extensible Markup Language

El *Extensible Markup Language* (XML) es un metalenguaje de marcado que permite representar datos jerárquicos de forma autodescriptiva mediante etiquetas definidas por el usuario [2]. XML fue el formato predominante en los primeros servicios web, especialmente en el ecosistema SOAP, donde su capacidad para describir esquemas complejos mediante XSD (*XML Schema Definition*) y la disponibilidad de mecanismos de validación formal lo convirtieron en la opción natural para entornos empresariales que exigen contratos de datos estrictos [1, 2].

El ecosistema XML comprende tecnologías complementarias que amplían significativamente su utilidad: XSLT permite transformar documentos XML en otros formatos; XPath y XQuery facilitan la navegación y consulta de estructuras jerárquicas; y los Namespaces resuelven los conflictos entre definiciones de elementos procedentes de esquemas diferentes [2]. No obstante, la verbosidad inherente a XML —que incrementa el tamaño de los mensajes y el costo de su procesamiento— ha limitado su adopción en entornos con restricciones de ancho de banda o con requisitos de baja latencia [13].

4.2. JSON: JavaScript Object Notation

El *JavaScript Object Notation* (JSON) es un formato de intercambio de datos ligero derivado de la sintaxis de objetos de JavaScript, que representa información estructurada mediante pares clave-valor y arrays anidados [11]. Su sintaxis minimalista, la facilidad de lectura humana y la eficiencia en los procesos de serialización y deserialización le han valido una adopción prácticamente universal en APIs RESTful, aplicaciones móviles y arquitecturas de microservicios. Estudios comparativos han evidenciado que los servicios basados en JSON presentan tiempos de procesamiento significativamente menores que sus equivalentes XML en escenarios de alta concurrencia [13].

JSON Schema proporciona un vocabulario para definir y validar la estructura de documentos JSON, acercando su nivel de rigor formal al de XSD en el ecosistema XML [11]. Existen también variantes orientadas a la eficiencia como BSON (Binary JSON) o MessagePack, diseñadas para escenarios donde la legibilidad no es prioritaria frente al rendimiento. La elección entre XML y JSON debe ponderar criterios de validación formal, interoperabilidad con sistemas legados, rendimiento bajo carga y familiaridad del equipo de desarrollo con las herramientas disponibles en cada ecosistema [1, 13].

XML	vs.	JSON
<pre> 1 <?xml version="1.0" encoding="UTF-8"?> 2 <endereco> 3 <cep>31270901</cep> 4 <city>Belo Horizonte</city> 5 <neighborhood>Pampulha</neighborhood> 6 <service>correios</service> 7 <state>MG</state> 8 <street>Av. Presidente Antônio Carlos, 6627</street> 9 </endereco> </pre>		<pre> 1 { 2 "endereco": { 3 "cep": "31270901", 4 "city": "Belo Horizonte", 5 "neighborhood": "Pampulha", 6 "service": "correios", 7 "state": "MG", 8 "street": "Av. Presidente Antônio Carlos, 6627" 9 } 10 } </pre>

Figura 3: Comparación de la representación de datos en formato XML y JSON, evidenciando la diferencia de verbosidad.

5. Arquitectura REST

Esta sección detalla los principios fundamentales de REST y los criterios de diseño de APIs RESTful. Dado que REST constituye el estilo arquitectónico predominante en el desarrollo moderno de servicios web, su comprensión profunda es esencial para la implementación de APIs correctas, mantenibles y escalables.

5.1. Fundamentos y restricciones del estilo REST

REST es un estilo arquitectónico para sistemas hipermedia distribuidos, formalizado por Roy T. Fielding en su disertación doctoral de 2000 [3]. No se trata de un protocolo ni de un estándar prescriptivo, sino de un conjunto de seis restricciones cuya aplicación conjunta proporciona propiedades deseables de escalabilidad, visibilidad, portabilidad y confiabilidad. Estas restricciones son: *cliente-servidor*, que separa las responsabilidades de interfaz de usuario y almacenamiento de datos; *sin estado* (stateless), que exige que cada solicitud contenga toda la información necesaria para su procesamiento; *caché*, que permite a los clientes y proxies intermedios almacenar respuestas para mejorar el rendimiento; *interfaz uniforme*, que simplifica y desacopla la arquitectura al estandarizar la forma de interacción; *sistema en capas*, que posibilita la inserción de intermediarios como proxies y pasarelas; y *código bajo demanda* (opcional), que permite al servidor transferir código ejecutable al cliente [3, 4].

La restricción de *interfaz uniforme* es la más definitoria de REST. Se concreta en cuatro subprincipios: la identificación de recursos mediante URIs únicas; la manipulación de recursos a través de representaciones; el uso de mensajes auto-descriptivos; y la *hipermedia como motor del estado de la aplicación* (HATEOAS), que permite al cliente descubrir dinámicamente las transiciones de estado disponibles a partir de los enlaces incluidos en las respuestas del servidor [4, 11].

5.2. Diseño de APIs RESTful

Una API RESTful bien diseñada organiza sus recursos en una jerarquía lógica de URIs que refleja las relaciones semánticas entre entidades, emplea los métodos HTTP con la semántica correcta y utiliza los códigos de estado HTTP para comunicar el resultado de cada operación de forma inequívoca [10, 11]. El versionado de las APIs —mediante prefijos en la URI (*/v1/*), cabeceras HTTP personalizadas o negociación de contenido— es una práctica recomendada para garantizar la compatibilidad hacia atrás cuando la interfaz evoluciona, permitiendo que los consumidores existentes continúen operando sin interrupciones [4].

La especificación OpenAPI se ha convertido en el estándar de facto para la descripción de APIs RESTful. A partir de un documento YAML o JSON que describe los endpoints, parámetros, esquemas de datos y mecanismos de autenticación, es posible generar automáticamente documentación interactiva, clientes en múltiples lenguajes y pruebas de contrato, lo que reduce el esfuerzo de integración y mejora la calidad de la documentación técnica [4, 11].

5.3. El modelo de madurez de Richardson

El modelo de madurez de Richardson describe cuatro niveles progresivos en la adopción de los principios REST. El nivel 0 utiliza HTTP como un mero túnel de transporte para llamadas RPC, sin aprovechar su semántica. El nivel 1 introduce recursos individuales identificados por URIs distintas. El nivel 2 incorpora el uso correcto de los verbos HTTP y los códigos de estado, logrando una interfaz predecible y semánticamente correcta. El nivel 3, el más avanzado, incorpora hipermedia (HATEOAS), permitiendo que las respuestas incluyan enlaces que describen las acciones disponibles sobre el recurso [4]. En la práctica industrial, la mayoría de las APIs comerciales se posicionan en el nivel 2, beneficiándose de la estandarización semántica de HTTP sin implementar la gestión de estado basada en hipermedia [11].

6. Arquitectura SOAP

Esta sección describe el protocolo SOAP, su estructura de mensajes, el rol del contrato WSDL y el ecosistema de extensiones WS-*. La comprensión de estos elementos permite valorar adecuadamente los contextos en que SOAP sigue siendo la alternativa más adecuada frente a REST.

6.1. Definición y estructura del protocolo

SOAP es un protocolo de intercambio de mensajes basado en XML diseñado para la comunicación entre aplicaciones en entornos distribuidos y heterogéneos [2, 14]. A diferencia de REST, SOAP define una estructura de mensaje formal denominada *sobre* (envelope), compuesta por tres partes. La cabecera (*Header*) es un elemento opcional que transporta metadatos de la transacción, como información de autenticación, trazas de auditoría o directivas de procesamiento. El cuerpo (*Body*) contiene el mensaje principal de la operación. El elemento de fallo (*Fault*), presente únicamente en respuestas de error, encapsula el código de error y la descripción textual, proporcionando un mecanismo estandarizado de reporte de errores [1, 14].

La independencia de transporte de SOAP permite su operación sobre múltiples protocolos subyacentes: HTTP, SMTP, TCP o colas de mensajes (MQ), lo que lo hace adecuado para arquitecturas de integración en las que la comunicación no puede restringirse exclusivamente a HTTP [2]. Esta flexibilidad lo distingue fundamentalmente de REST, que asume HTTP como protocolo de aplicación.

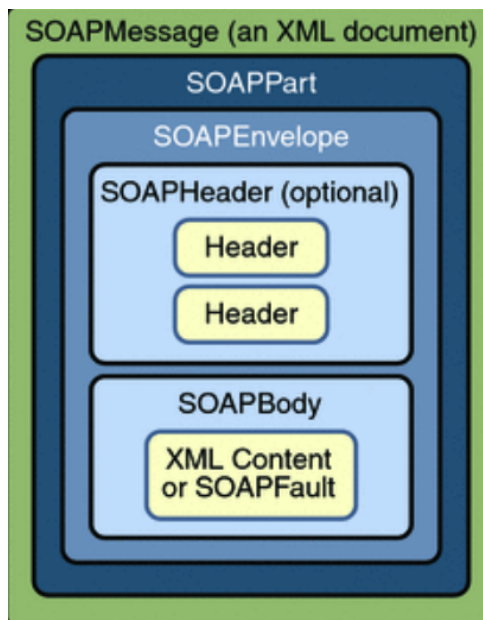


Figura 4: Estructura del mensaje SOAP mostrando los componentes Envelope, Header, Body y Fault.

6.2. WSDL y el contrato de servicios

El *Web Services Description Language* (WSDL) es el lenguaje estándar empleado para describir de forma completa y formal los servicios SOAP [1]. Un documento WSDL especifica los *tipos* de datos empleados (normalmente mediante XSD), los *mensajes* de entrada y salida, las *operaciones* agrupadas en interfaces (*portTypes*), los *bindings* que asocian una interfaz con un protocolo de transporte y los *servicios* que indican la dirección del endpoint. Esta descripción formal actúa como un contrato inequívoco entre proveedor y consumidor, y facilita la generación automática de proxies de cliente mediante herramientas como Apache Axis2 o JAX-WS [2, 15].

6.3. El ecosistema WS-*

Una de las principales fortalezas de SOAP reside en su ecosistema de extensiones estandarizadas, denominadas colectivamente WS-*, que abordan aspectos críticos para entornos empresariales. WS-Security define cómo aplicar firmas digitales XML y cifrado a nivel de mensaje, garantizando la seguridad de los datos incluso cuando el mensaje atraviesa intermediarios que terminan la conexión TLS [1, 15]. WS-ReliableMessaging asegura la entrega ordenada y sin duplicados de mensajes en redes no fiables. WS-AtomicTransaction y WS-BusinessActivity proporcionan soporte para transacciones distribuidas de corta y larga duración, respectivamente. WS-Addressing permite el enrutamiento de mensajes de forma independiente de la capa de transporte, facilitando arquitecturas de mensajería

asíncrona. La madurez y amplitud de este ecosistema explica la continuidad de SOAP como estándar dominante en sectores como servicios financieros, sanidad e integración gubernamental [2, 15].

7. Comparación entre REST y SOAP

Esta sección presenta un contraste orientado a decisiones de diseño entre REST y SOAP, considerando criterios técnicos y no funcionales. El propósito no es establecer un ganador universal, sino ofrecer un marco de comparación que permita justificar una selección tecnológica en función del contexto y los requisitos del sistema.

7.1. Criterios de análisis

La elección entre REST y SOAP constituye una decisión arquitectónica dependiente del contexto, los requisitos no funcionales y las restricciones operativas del sistema [1, 4]. Desde el punto de vista de la complejidad técnica, SOAP introduce una capa adicional de especificación mediante el uso de sobres XML y contratos WSDL formales, lo que incrementa la curva de aprendizaje y el esfuerzo de desarrollo. En contraste, REST aprovecha la semántica nativa de HTTP, reduciendo la cantidad de conceptos adicionales que deben dominarse [11, 13].

En términos de rendimiento, la verbosidad inherente a los mensajes SOAP implica un mayor consumo de ancho de banda y recursos de procesamiento. Diversos estudios han evidenciado que las APIs REST basadas en JSON presentan mejores tiempos de respuesta en escenarios de alta concurrencia con intercambios de datos relativamente simples [13].

7.2. Fortalezas y limitaciones de cada enfoque

REST sobresale por su agilidad de desarrollo, su integración natural con tecnologías web y móviles, y su compatibilidad con mecanismos de caché HTTP [3, 4]. No obstante, carece de estándares nativos para la gestión de transacciones distribuidas y la seguridad a nivel de mensaje, aspectos en los que SOAP dispone de soluciones maduras y formalmente estandarizadas [1, 15].

SOAP, por su parte, resulta especialmente adecuado en entornos empresariales donde se requieren contratos estrictos de datos, validación formal mediante XSD, seguridad a nivel de mensaje y soporte para transacciones distribuidas [1, 2]. La Tabla 1 sintetiza los principales criterios de comparación entre ambos enfoques.

Cuadro 1: Comparación entre REST y SOAP

Criterio	REST	SOAP
Protocolo base	HTTP/HTTPS	HTTP, SMTP, TCP, MQ
Formato de mensajes	JSON, XML, texto	XML obligatorio
Descripción de interfaz	OpenAPI (opcional)	WSDL (formal)
Gestión de estado	Sin estado	Sin estado / con estado

Criterio	REST	SOAP
Seguridad	TLS (HTTPS)	WS-Security + TLS
Transacciones	No nativo	WS-Transaction
Curva de aprendizaje	Baja a media	Alta
Casos de uso típicos	APIs web, móviles, micro-servicios	Integración B2B, finanzas, salud

8. APIs y servicios externos

Esta sección contextualiza el rol de las APIs como interfaz de integración en sistemas distribuidos y describe los patrones más relevantes para su consumo externo. La comprensión de la economía de APIs y de los patrones de integración es fundamental para diseñar sistemas extensibles y resilientes frente a dependencias de terceros.

8.1. Concepto y taxonomía de las APIs

Una *Application Programming Interface* (API) es un conjunto de definiciones, protocolos y herramientas que establece cómo los componentes de software deben interactuar entre sí, actuando como capa de abstracción entre la implementación interna y los consumidores externos [16]. En el contexto de los servicios web, las APIs se clasifican en tres categorías principales: las APIs *privadas*, accesibles únicamente dentro de la organización que las desarrolla; las APIs *de socio* (partner), compartidas con actores externos bajo acuerdos contractuales; y las APIs *públicas*, disponibles para cualquier desarrollador registrado a través de portales de autoservicio [16].

Las APIs públicas de plataformas como Google Cloud, Meta, Stripe o Firebase han transformado el ecosistema del desarrollo de software al permitir que aplicaciones de terceros integren capacidades complejas —cartografía, autenticación, pagos, almacenamiento en tiempo real— sin necesidad de replicar la infraestructura subyacente [16, 17]. Esta economía de APIs ha convertido la capacidad de exposición y consumo de interfaces programáticas en un activo estratégico para las organizaciones.

8.2. Integración de APIs externas: patrones y consideraciones

La integración de APIs externas en sistemas distribuidos introduce dependencias de disponibilidad y estabilidad que deben ser gestionadas explícitamente mediante patrones de resiliencia. El patrón *circuit breaker* evita la propagación de fallos en cascada interrumpiendo temporalmente las solicitudes a un servicio externo que ha comenzado a fallar, permitiendo que el sistema se recupere mientras el cliente aplica una respuesta alternativa [5, 9]. Complementariamente, las estrategias de *retry* con retroceso exponencial y *jitter* reducen la presión sobre servicios que atraviesan períodos de sobrecarga transitoria [9].

La centralización de la lógica de integración en pasarelas de API (*API gateways*) simplifica la gestión de autenticación, autorización, limitación de tasas, transformación de formatos y registro de auditoría, sin contaminar la lógica de negocio de los servicios individuales [5, 6]. El patrón *Backend for Frontend* (BFF) crea pasarelas especializadas para cada tipo de cliente (web, móvil, IoT), reduciendo la sobre-obtención o sub-obtención

de datos y adaptando la interfaz a las necesidades específicas de cada canal de presentación [6]. La gestión del versionado y los cambios incompatibles (*breaking changes*) en las APIs de terceros, así como el diseño de mecanismos de *fallback*, son consideraciones críticas para garantizar la continuidad operativa del sistema integrador [5, 16].

9. Autenticación y autorización: OAuth 2.0

Esta sección desarrolla los mecanismos de seguridad de identidad más relevantes para los servicios web modernos. La correcta implementación de OAuth 2.0 y OpenID Connect es condición indispensable para construir APIs seguras y usables, tanto en entornos de usuario final como en comunicaciones máquina a máquina.

9.1. Fundamentos de la seguridad de identidad

La autenticación es el proceso de verificar la identidad de un actor (usuario, cliente, servicio), mientras que la autorización determina qué recursos y operaciones están permitidos para una identidad autenticada [18, 19]. En el contexto de los servicios web, la separación clara de ambos conceptos es fundamental para el diseño de sistemas seguros y constituye la base del modelo de delegación de autorización implementado por OAuth 2.0. Los enfoques tradicionales basados en sesiones y cookies presentan limitaciones significativas en arquitecturas sin estado y multi-dominio, lo que ha impulsado la adopción de tokens portadores (*bearer tokens*) y mecanismos federados de identidad [19].

9.2. OAuth 2.0: marco de autorización

OAuth 2.0 es un protocolo de autorización estandarizado en el RFC 6749 que permite a una aplicación (cliente) obtener acceso limitado a recursos protegidos en nombre de un propietario de recursos, sin necesidad de que el cliente acceda directamente a las credenciales del usuario [18]. El protocolo define cuatro roles: el propietario del recurso, el servidor de recursos, el cliente y el servidor de autorización; y varios flujos de autorización adaptados a diferentes escenarios: *Authorization Code*, *Client Credentials*, *Device Authorization* e *Implicit* (este último desaconsejado en versiones recientes) [18, 19].

El flujo *Authorization Code con PKCE* (Proof Key for Code Exchange) es el recomendado para aplicaciones públicas como clientes móviles y SPAs, ya que mitiga los ataques de interceptación de código de autorización sin requerir un secreto de cliente [19]. Los tokens de acceso en implementaciones modernas suelen tomar la forma de *JSON Web Tokens* (JWT), firmados digitalmente con algoritmos asimétricos que permiten su validación sin necesidad de consultar al servidor de autorización en cada solicitud [19].

9.3. OpenID Connect

OpenID Connect (OIDC) es una capa de identidad construida sobre OAuth 2.0 que añade autenticación al mecanismo de autorización, proporcionando un token de identidad (ID Token) con información verificable sobre el usuario autenticado [20]. El ID Token es un JWT firmado que contiene *claims* estandarizados como el identificador del sujeto, el emisor, la audiencia y el tiempo de expiración, que permiten verificar su autenticidad y vigencia sin comunicación adicional con el servidor. OIDC es ampliamente adoptado por proveedores de identidad empresariales como Google, Microsoft Azure AD y Okta, y su

combinación con OAuth 2.0 constituye la solución de facto para la gestión de identidades en APIs modernas [19, 20].

10. Servicios en la nube

Esta sección describe los modelos de servicio en la nube y las plataformas más relevantes para el despliegue de servicios web. La computación en la nube ha transformado la economía del despliegue de software, haciendo posible la escalabilidad elástica y la disponibilidad global a costos accesibles para organizaciones de cualquier tamaño.

10.1. Modelos de servicio: IaaS, PaaS y SaaS

La computación en la nube puede definirse como un modelo para habilitar el acceso conveniente, bajo demanda y a través de la red a un conjunto compartido de recursos de cómputo configurables que se pueden aprovisionar y liberar con mínimo esfuerzo de gestión [21]. El NIST identifica tres modelos de servicio: *Infrastructure as a Service* (IaaS), en el que el proveedor ofrece recursos de infraestructura virtualizados; *Platform as a Service* (PaaS), que añade un entorno de ejecución gestionado con herramientas de desarrollo y middleware; y *Software as a Service* (SaaS), en el que el proveedor entrega aplicaciones completas accesibles a través del navegador [21, 22].

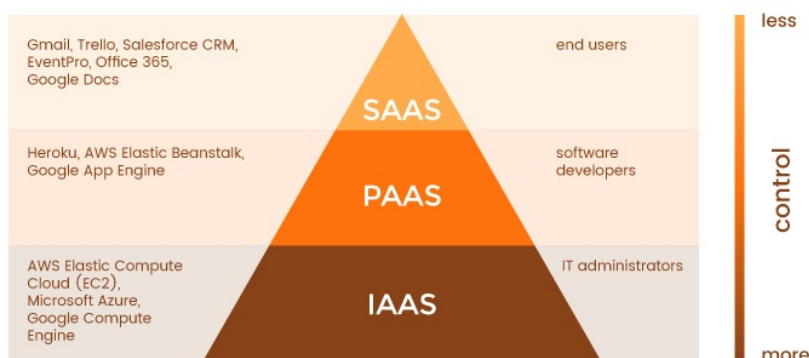


Figura 5: Modelos de servicio en la nube: Infrastructure as a Service (IaaS), Platform as a Service (PaaS) y Software as a Service (SaaS).

Adicionalmente, el NIST define cuatro modelos de despliegue: nube pública, privada, comunitaria e híbrida. La nube híbrida, que combina infraestructura propia con servicios de proveedores públicos, ha ganado adopción en organizaciones que deben conciliar requisitos de cumplimiento normativo con la necesidad de escalabilidad elástica [21, 22].

10.2. Plataformas y APIs relevantes

Los principales proveedores de servicios en la nube —Amazon Web Services (AWS), Google Cloud Platform (GCP) y Microsoft Azure— ofrecen catálogos de servicios gestionados que abarcan cómputo, almacenamiento, bases de datos, mensajería, inteligencia artificial y herramientas de integración continua [17]. En el ámbito del desarrollo de aplicaciones web, Firebase destaca por su base de datos en tiempo real, servicios de autenticación, funciones serverless y hosting, todos accesibles mediante APIs REST y SDK

nativos. Por su parte, Heroku simplifica el despliegue mediante una capa de abstracción sobre AWS, ofreciendo una API REST para la gestión de sus componentes [17].

El paradigma *serverless* (o FaaS, Functions as a Service), representado por AWS Lambda, Google Cloud Functions y Azure Functions, ha emergido como una alternativa al despliegue de servicios tradicionales, permitiendo la ejecución de funciones sin gestión explícita de servidores y con facturación por invocación [22]. Este modelo favorece la escalabilidad automática y reduce los costos operativos en cargas de trabajo irregulares, aunque introduce consideraciones de rendimiento relacionadas con el tiempo de inicio en frío (*cold start*) de las funciones.

11. Escalabilidad y rendimiento en servicios web

Esta sección examina los principios y patrones que permiten diseñar servicios web capaces de mantener un desempeño aceptable bajo condiciones de carga creciente. La escalabilidad y el rendimiento deben considerarse desde las decisiones iniciales de arquitectura y no como atributos que puedan añadirse de forma retroactiva.

11.1. Escalabilidad horizontal y vertical

La escalabilidad es la capacidad de un sistema de mantener o mejorar su nivel de desempeño cuando la carga de trabajo aumenta [23]. La *escalabilidad vertical* (scale-up) consiste en incrementar los recursos de un nodo existente (CPU, RAM, almacenamiento), mientras que la *escalabilidad horizontal* (scale-out) implica la adición de nuevos nodos al sistema. La escalabilidad horizontal es generalmente preferida en arquitecturas de servicios web modernos por su mayor flexibilidad, resiliencia y relación costo-beneficio, ya que no está limitada por el hardware máximo disponible para una sola máquina [7, 23].

El modelo *AKF Scale Cube* describe tres ejes de escalabilidad: el eje X (clonación de instancias), el eje Y (descomposición por funcionalidad, que da origen a los microservicios) y el eje Z (particionamiento de datos por cliente o segmento) [23]. La combinación de los tres ejes permite diseñar sistemas capaces de escalar a millones de usuarios sin degradación del rendimiento.

11.2. Patrones de rendimiento y optimización

La caché es uno de los mecanismos de optimización más efectivos en servicios web; su correcta implementación —mediante cabeceras HTTP como **Cache-Control**, **ETag** y **Last-Modified**— puede reducir drásticamente la carga sobre los servidores de origen [3, 10]. Las redes de entrega de contenidos (CDN) complementan la caché al distribuir geográficamente los recursos cerca de los usuarios finales, reduciendo la latencia de red.

El *balanceo de carga* distribuye las solicitudes entrantes entre múltiples instancias del servicio mediante algoritmos como round-robin, least-connections o hash consistente, garantizando tanto la distribución equitativa de la carga como la afinidad de sesión cuando es necesaria [7]. La adopción de protocolos de comunicación asíncrona —como colas de mensajes (RabbitMQ, Apache Kafka) y el patrón *event-driven architecture*— permite desacoplar los productores de los consumidores, absorber picos de demanda y facilitar la auditabilidad mediante el registro de eventos inmutables [8, 24].

12. Seguridad en servicios web

Esta sección integra los conocimientos de secciones anteriores para construir un modelo de seguridad holístico aplicable a servicios web y APIs. La seguridad debe concebirse como una propiedad transversal del sistema y no como un módulo adicional, dado que las vulnerabilidades pueden manifestarse en cualquiera de las capas de la arquitectura.

12.1. Amenazas principales

La seguridad de los servicios web es un campo multidimensional que abarca la protección de la capa de transporte, la autenticación de las partes comunicantes, la autorización granular de operaciones y la validación de los datos intercambiados. El *OWASP API Security Top 10* cataloga las vulnerabilidades más críticas en APIs web: autorización a nivel de objeto rota (BOLA/IDOR), autenticación comprometida, exposición excesiva de datos, falta de limitación de recursos, escasez de registro y monitoreo, y falsificación de solicitudes del lado del servidor (SSRF), entre otras [25].

Los ataques de inyección —SQL injection, NoSQL injection, XML injection y command injection— son igualmente relevantes cuando el servicio web construye consultas a partir de datos de entrada sin la validación y el saneamiento adecuados [25, 26]. La gestión incorrecta de tokens de autorización, como el almacenamiento en `localStorage` de SPAs o la ausencia de rotación de tokens de larga vida, constituye una fuente frecuente de incidentes de seguridad en implementaciones OAuth 2.0 [19].

12.2. Buenas prácticas de seguridad

La mitigación de las amenazas anteriores requiere la implementación de múltiples capas de protección coordinadas. En la capa de transporte, el uso obligatorio de TLS 1.2 o superior con suites criptográficas seguras es condición necesaria pero no suficiente [12]. En la capa de aplicación, la validación estricta de entradas mediante listas de permitidos, la implementación de limitación de tasas (*rate limiting*), el uso de tokens JWT de corta duración con mecanismos de revocación, y la aplicación del principio de mínimo privilegio en el diseño de los permisos son prácticas esenciales [19, 25].

La implementación de una API Gateway centraliza la aplicación de políticas transversales de seguridad —autenticación, autorización, limitación de tasas, registro de auditoría— sin contaminar la lógica de negocio de los microservicios individuales [5, 6]. El monitoreo continuo mediante la correlación de registros de acceso y la detección de anomalías en el tráfico de la API, apoyado en soluciones SIEM (*Security Information and Event Management*), completa el ciclo de seguridad defensiva y permite la respuesta temprana ante incidentes [26].

13. Conclusiones

En esta sección se sintetizan los hallazgos del marco teórico y se consolidan las ideas que sustentan la selección de enfoques, mecanismos de seguridad y consideraciones de escalabilidad. La intención es cerrar el apartado bibliográfico conectando los conceptos desarrollados con criterios aplicables a la implementación en sistemas distribuidos.

El análisis de los servicios web, las arquitecturas REST y SOAP, y la integración de APIs externas en sistemas distribuidos evidencia un dominio tecnológico en constante evolución, en el que los principios fundamentales —interoperabilidad, escalabilidad, seguridad e independencia de plataforma— permanecen vigentes a pesar de los cambios en las herramientas y enfoques de implementación.

La arquitectura REST, sustentada en la semántica de HTTP y el uso de formatos ligeros como JSON, se ha consolidado como el paradigma predominante para el desarrollo de APIs públicas y microservicios debido a su flexibilidad y eficiencia. SOAP, en contraste, mantiene su relevancia en contextos empresariales de alta criticidad, donde los contratos formales, la seguridad a nivel de mensaje y las transacciones distribuidas son requisitos esenciales. La comprensión profunda de ambos enfoques resulta indispensable para el diseño de soluciones de integración robustas.

Finalmente, los mecanismos de autenticación y autorización basados en OAuth 2.0 y OpenID Connect, junto con las capacidades de la computación en la nube, proporcionan la base para la construcción de sistemas distribuidos seguros y escalables. La seguridad, entendida como una propiedad transversal del sistema, debe integrarse desde las fases iniciales del diseño para garantizar la confiabilidad y sostenibilidad de las arquitecturas modernas.

Actividad práctica: Implementación y consumo de servicios web escalables (REST y SOAP)

La presente práctica experimental tiene como objetivo implementar y consumir servicios web escalables utilizando las arquitecturas REST y SOAP, integrando APIs públicas externas y aplicando buenas prácticas de seguridad, rendimiento e interoperabilidad en sistemas distribuidos.

14. Objetivos de la práctica

A continuación se detallan los objetivos generales y específicos que guiaron el desarrollo de esta práctica experimental.

14.1. Objetivo general

Diseñar, implementar y consumir servicios web escalables utilizando las arquitecturas RESTful y SOAP, e integrar APIs públicas como Google API, Facebook API, Firebase y Heroku, aplicando buenas prácticas de seguridad, rendimiento e interoperabilidad.

14.2. Objetivos específicos

- Implementar una API REST completa con operaciones CRUD utilizando Spring Boot
- Desarrollar un servicio SOAP con generación automática de WSDL
- Integrar servicios externos: Facebook Graph API y Firebase Realtime Database
- Realizar pruebas de concurrencia y rendimiento para comparar REST vs SOAP
- Documentar la arquitectura del sistema implementado

14.3. Alcance

Esta práctica abarca la implementación de servicios web propios mediante REST y SOAP, la integración con tres APIs externas gratuitas, la ejecución de pruebas de rendimiento bajo carga concurrente en un entorno local y la documentación técnica completa del sistema desarrollado. El alcance comprende las etapas principales del desarrollo de servicios web, desde el diseño de la arquitectura hasta las pruebas y la documentación final.

El sistema desarrollado evidencia la aplicación de conceptos fundamentales como la separación de responsabilidades, la inyección de dependencias y el uso de patrones de diseño como Repository, Service Layer y DTO. Además, se aplicaron tecnologías modernas de desarrollo backend como Spring Framework, JPA/Hibernate y el consumo de APIs externas mediante clientes HTTP.

Aunque el sistema fue implementado con fines académicos en un entorno local, se siguieron estándares y buenas prácticas que permiten su adaptación a entornos de producción, requiriendo únicamente ajustes relacionados con seguridad, escalabilidad y disponibilidad.

15. Tecnologías y herramientas

En esta sección se describen todas las tecnologías, frameworks y herramientas utilizadas durante el desarrollo de la práctica. La selección de estas tecnologías se basó en criterios de madurez, documentación disponible, compatibilidad y relevancia en el contexto profesional actual.

15.1. Entorno de desarrollo

- **Framework:** Spring Boot 3.5.10
- **Lenguaje:** Java 17
- **Build Tool:** Maven 3.9.x
- **IDE:** IntelliJ IDEA
- **Sistema Operativo:** Windows 11
- **Hardware:** AMD Ryzen 5, 16GB RAM

15.2. Tecnologías backend

- **Spring Web:** Para implementación de controladores REST
- **Spring Web Services:** Para implementación de servicios SOAP
- **Spring Data JPA:** Para persistencia de datos
- **PostgreSQL:** Base de datos relacional (versión 15)
- **RestClient:** Para consumo de APIs externas
- **JAXB:** Para generación de clases Java desde XSD

15.3. APIs externas integradas

- **OpenStreetMap Nominatim (alternativa gratuita a Google Maps):** Servicios de geocodificación
- **Facebook Graph API v19.0:** Autenticación y datos de perfil
- **Firebase Realtime Database:** Base de datos NoSQL en tiempo real

15.4. Herramientas de prueba

- **Postman:** Testing de APIs y pruebas de concurrencia
- **Postman Runner:** Ejecución de pruebas de carga
- **pgAdmin 4:** Administración de PostgreSQL

16. Arquitectura del sistema

Este capítulo describe la arquitectura del sistema implementado, incluyendo el diseño en capas, los patrones de diseño aplicados y la interacción entre componentes. La arquitectura fue diseñada siguiendo principios SOLID y buenas prácticas de desarrollo de software empresarial.

16.1. Diseño arquitectónico

La aplicación implementada sigue una arquitectura en capas basada en el patrón MVC (Model-View-Controller) adaptado para servicios web. La arquitectura se compone de las siguientes capas principales:

16.1.1. Capa de presentación

- **REST Controllers:** Exponen endpoints RESTful bajo la ruta `/api/v1/*` utilizando formato JSON. Implementan los métodos HTTP estándar (GET, POST, PUT, DELETE) con códigos de estado apropiados.
- **SOAP Endpoint:** Expone servicio SOAP bajo la ruta `/ws/*` utilizando formato XML con WSDL autogenerado y validación mediante XSD.

16.1.2. Capa de lógica de negocio

- **Service Layer:** Contiene la lógica de negocio y reglas de validación. Desacopla la presentación del acceso a datos mediante inyección de dependencias.

16.1.3. Capa de acceso a datos

- **Repository Layer:** Implementa el patrón Repository usando Spring Data JPA. Abstrae las operaciones CRUD sobre PostgreSQL y proporciona métodos de consulta personalizados.

16.1.4. Capa de persistencia

- **PostgreSQL:** Base de datos relacional que almacena las entidades del sistema (productos) con sus respectivas relaciones.

16.1.5. Integraciones externas

- **OpenStreetMap Nominatim:** Servicios de geocodificación directa e inversa
- **Facebook Graph API:** Autenticación OAuth 2.0 y obtención de datos de perfil
- **Firebase Realtime Database:** Almacenamiento NoSQL en tiempo real con sincronización automática

16.2. Diagrama de arquitectura

La Figura 6 muestra la arquitectura completa del sistema implementado, incluyendo todos los componentes y sus interacciones.

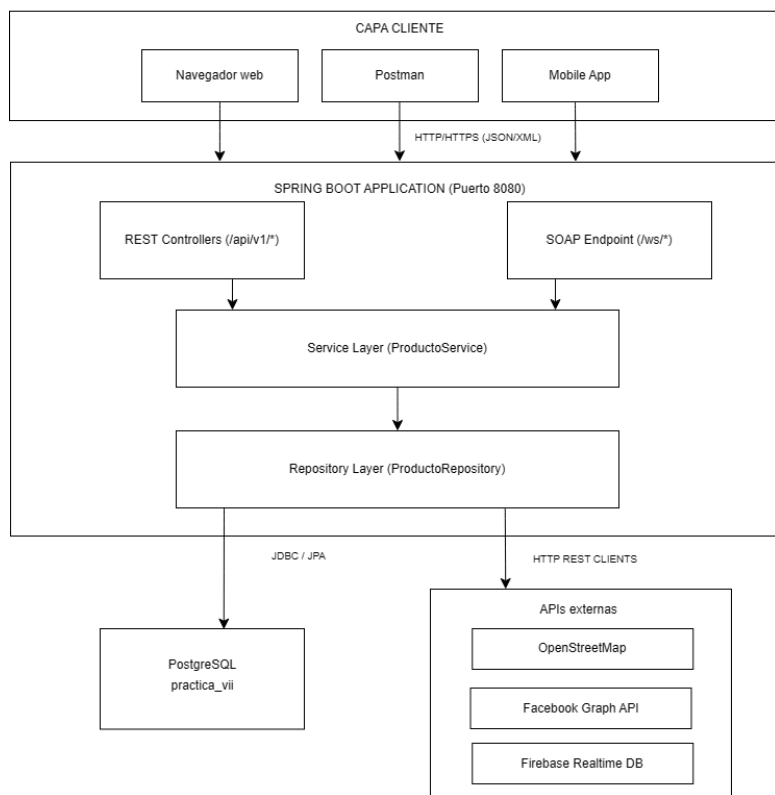


Figura 6: Arquitectura del sistema de servicios web

16.3. Patrones de diseño aplicados

- **Repository Pattern:** Abstracción del acceso a datos
- **Dependency Injection:** Gestión de dependencias mediante Spring IoC
- **DTO Pattern:** Transferencia de datos entre capas (implícito en entidades JPA)
- **REST API Design:** Siguiendo principios RESTful y convenciones HTTP
- **Service Layer Pattern:** Separación de lógica de negocio

17. Implementación de servicios web

Este capítulo presenta los detalles técnicos de la implementación, incluyendo la configuración del proyecto, el código fuente de los componentes principales y las decisiones de diseño tomadas durante el desarrollo. Se incluyen fragmentos de código relevantes para ilustrar la implementación de cada funcionalidad.

17.1. Configuración del proyecto

La configuración inicial del proyecto es fundamental para establecer las dependencias y propiedades necesarias. A continuación se describen los archivos de configuración principales.

17.1.1. Dependencias Maven

El proyecto utiliza las siguientes dependencias principales en `pom.xml`:

```
1 <dependencies>
2   <!-- REST -->
3   <dependency>
4     <groupId>org.springframework.boot</groupId>
5     <artifactId>spring-boot-starter-web</artifactId>
6   </dependency>
7
8   <!-- SOAP -->
9   <dependency>
10    <groupId>org.springframework.boot</groupId>
11    <artifactId>spring-boot-starter-web-services</artifactId>
12  </dependency>
13
14  <!-- JPA + PostgreSQL -->
15  <dependency>
16    <groupId>org.springframework.boot</groupId>
17    <artifactId>spring-boot-starter-data-jpa</artifactId>
18  </dependency>
19  <dependency>
20    <groupId>org.postgresql</groupId>
21    <artifactId>postgresql</artifactId>
22  </dependency>
23
24  <!-- Cliente HTTP -->
25  <dependency>
26    <groupId>org.springframework.boot</groupId>
27    <artifactId>spring-boot-starter-webflux</artifactId>
28  </dependency>
29 </dependencies>
```

Listing 1: Dependencias principales del proyecto

17.1.2. Configuración de la aplicación

El archivo `application.properties` contiene la configuración de la base de datos y las APIs externas:

```

1 # Aplicacion
2 spring.application.name=practica-web-services
3 server.port=8080
4
5 # PostgreSQL
6 spring.datasource.url=jdbc:postgresql://localhost:5432/practica_vii
7 spring.datasource.username=postgres
8 spring.datasource.password=postgresAdmin19
9 spring.datasource.driver-class-name=org.postgresql.Driver
10
11 # JPA/Hibernate
12 spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
13 spring.jpa.hibernate.ddl-auto=update
14 spring.jpa.show-sql=true
15
16 # FACEBOOK API
17 facebook.access.token=EAAMZBk05Jk6UBQn0bZCaZA1In7rr...
18
19 # FIREBASE REALTIME DATABASE
20 firebase.database.url=https://uteq-practica-web-services-default-rtdb.firebaseio.com

```

Listing 2: Configuración de `application.properties`

17.2. Implementación de API REST

La API REST implementa un CRUD completo sobre la entidad `Producto`, siguiendo las convenciones HTTP y principios RESTful. A continuación se presenta la implementación de cada capa.

17.2.1. Modelo de datos

Se implementó la entidad `Producto` como modelo de datos principal:

```

1 @Entity
2 @Data
3 public class Producto {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String nombre;
9     private String descripcion;
10    private Double precio;
11    private Integer stock;
12 }

```

Listing 3: Entidad `Producto`

17.2.2. Capa de repositorio

Utilizando Spring Data JPA, se creó la interfaz de repositorio:

```
1 public interface ProductoRepository
2     extends JpaRepository<Producto, Long> {
3 }
```

Listing 4: Repository de Producto

17.2.3. Capa de servicio

La lógica de negocio se implementó en la clase de servicio:

```
1 @Service
2 public class ProductoService {
3     private final ProductoRepository repository;
4
5     public List<Producto> listarTodos() {
6         return repository.findAll();
7     }
8
9     public Optional<Producto> buscarPorId(Long id) {
10        return repository.findById(id);
11    }
12
13    public Producto crear(Producto producto) {
14        return repository.save(producto);
15    }
16
17    public Producto actualizar(Long id, Producto producto) {
18        producto.setId(id);
19        return repository.save(producto);
20    }
21
22    public void eliminar(Long id) {
23        repository.deleteById(id);
24    }
25 }
```

Listing 5: Service de Producto

17.2.4. Controlador REST

Se implementaron los endpoints RESTful siguiendo las convenciones HTTP:

```
1 @RestController
2 @RequestMapping("/api/v1/productos")
3 public class ProductoRestController {
4
5     private final ProductoService service;
6
7     @GetMapping
8     public ResponseEntity<List<Producto>> listarTodos() {
9         return ResponseEntity.ok(service.listarTodos());
10    }
11
12    @GetMapping("/{id}")
```

```

13 public ResponseEntity<Producto> buscarPorId(@PathVariable Long id) {
14     return service.buscarPorId(id)
15         .map(ResponseEntity::ok)
16         .orElse(ResponseEntity.notFound().build());
17 }
18
19 @PostMapping
20 public ResponseEntity<Producto> crear(@RequestBody Producto producto) {
21     Producto nuevo = service.crear(producto);
22     return ResponseEntity.status(HttpStatus.CREATED).body(nuevo);
23 }
24
25 @PutMapping("/{id}")
26 public ResponseEntity<Producto> actualizar(
27     @PathVariable Long id,
28     @RequestBody Producto producto) {
29     return ResponseEntity.ok(service.actualizar(id, producto));
30 }
31
32 @DeleteMapping("/{id}")
33 public ResponseEntity<Void> eliminar(@PathVariable Long id) {
34     service.eliminar(id);
35     return ResponseEntity.noContent().build();
36 }
37 }

```

Listing 6: REST Controller

17.3. Implementación de servicio SOAP

El servicio SOAP proporciona una alternativa basada en XML para el acceso a los datos. Esta implementación demuestra la integración de ambas arquitecturas (REST y SOAP) en un mismo sistema.

17.3.1. Definición del schema XSD

Se definió el contrato del servicio mediante un schema XML:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
3     xmlns:tns="http://uteq.com/practicawebsservices/productos"
4     targetNamespace="http://uteq.com/practicawebsservices/productos"
5     elementFormDefault="qualified">
6
7     <xs:element name="getProductoRequest">
8         <xs:complexType>
9             <xs:sequence>
10                 <xs:element name="id" type="xs:long"/>
11             </xs:sequence>
12         </xs:complexType>
13     </xs:element>
14
15     <xs:element name="getProductoResponse">
16         <xs:complexType>
17             <xs:sequence>
18                 <xs:element name="producto" type="tns:producto"/>
19             </xs:sequence>

```



```

20     </xs:complexType>
21 </xs:element>
22
23 <xs:complexType name="producto">
24     <xs:sequence>
25         <xs:element name="id" type="xs:long"/>
26         <xs:element name="nombre" type="xs:string"/>
27         <xs:element name="descripcion" type="xs:string"/>
28         <xs:element name="precio" type="xs:double"/>
29         <xs:element name="stock" type="xs:int"/>
30     </xs:sequence>
31 </xs:complexType>
32 </xs:schema>

```

Listing 7: Schema XSD para SOAP

17.3.2. Endpoint SOAP

Se implementó el endpoint SOAP con mapeo de objetos JPA a objetos JAXB:

```

1 @Endpoint
2 public class ProductoEndpoint {
3     private static final String NAMESPACE_URI =
4         "http://uteq.com/practicawebservices/productos";
5
6     private final ProductoService service;
7
8     @PayloadRoot(namespace = NAMESPACE_URI,
9         localPart = "getProductoRequest")
10    @ResponsePayload
11    public GetProductoResponse getProducto(
12        @RequestPayload GetProductoRequest request) {
13        GetProductoResponse response = new GetProductoResponse();
14
15        service.buscarPorId(request.getId())
16            .ifPresent(productoJpa -> {
17                com.uteq.practicawebservices.productos.Producto
18                    productoSoap = new com.uteq...Producto();
19                productoSoap.setId(productoJpa.getId());
20                productoSoap.setNombre(productoJpa.getNombre());
21                productoSoap.setDescripcion(productoJpa.getDescripcion());
22                productoSoap.setPrecio(productoJpa.getPrecio());
23                productoSoap.setStock(productoJpa.getStock());
24                response.setProducto(productoSoap);
25            });
26
27        return response;
28    }
29 }

```

Listing 8: SOAP Endpoint

17.3.3. Configuración SOAP

Se configuró el servlet SOAP y la generación automática del WSDL:

```
1 @EnableWs
2 @Configuration
3 public class WebServiceConfig {
4
5     @Bean
6     public ServletRegistrationBean<MessageDispatcherServlet>
7         messageDispatcherServlet(ApplicationContext context) {
8         MessageDispatcherServlet servlet =
9             new MessageDispatcherServlet();
10        servlet.setApplicationContext(context);
11        servlet.setTransformWsdLocations(true);
12        return new ServletRegistrationBean<>(servlet, "/ws/*");
13    }
14
15    @Bean(name = "productos")
16    public DefaultWsd11Definition defaultWsd11Definition(
17        XsdSchema productosSchema) {
18        DefaultWsd11Definition definition =
19            new DefaultWsd11Definition();
20        definition.setPortTypeName("ProductosPort");
21        definition.setLocationUri("/ws");
22        definition.setTargetNamespace(
23            "http://uteq.com/practicaweb/services/productos");
24        definition.setSchema(productosSchema);
25        return definition;
26    }
27
28    @Bean
29    public XsdSchema productosSchema() {
30        return new SimpleXsdSchema(
31            new ClassPathResource("xsd/productos.xsd"));
32    }
33 }
```

Listing 9: Configuración de Web Services

17.4. Integración con APIs externas

La integración con servicios externos permite ampliar la funcionalidad del sistema sin necesidad de implementar todas las características desde cero. A continuación se presentan las tres integraciones realizadas.

17.4.1. OpenStreetMap Nominatim

Se implementó un controlador para servicios de geocodificación:

```

1 @RestController
2 @RequestMapping("/api/v1/openstreetmap")
3 public class OpenStreetMapController {
4
5     private final RestClient restClient = RestClient.builder()
6         .baseUrl("https://nominatim.openstreetmap.org")
7         .defaultHeader("User-Agent", "UTEQ-Practica/1.0")
8         .build();
9
10    @GetMapping("/geocode")
11    public String geocodificarDireccion(
12        @RequestParam String address) {
13        return restClient.get()
14            .uri(uriBuilder -> uriBuilder
15                .path("/search")
16                .queryParams("q", address)
17                .queryParams("format", "json")
18                .queryParams("limit", 5)
19                .build())
20            .retrieve()
21            .body(String.class);
22    }
23
24    @GetMapping("/reverse")
25    public String geocodificarInverso(
26        @RequestParam Double lat,
27        @RequestParam Double lon) {
28        return restClient.get()
29            .uri(uriBuilder -> uriBuilder
30                .path("/reverse")
31                .queryParams("lat", lat)
32                .queryParams("lon", lon)
33                .queryParams("format", "json")
34                .build())
35            .retrieve()
36            .body(String.class);
37    }
38 }

```

Listing 10: Controller de OpenStreetMap

17.4.2. Facebook Graph API

Se implementó la integración con autenticación OAuth 2.0:

```
1 @RestController
2 @RequestMapping("/api/v1/facebook")
3 public class FacebookApiController {
4
5     @Value("${facebook.access.token}")
6     private String defaultAccessToken;
7
8     private final RestClient restClient = RestClient.builder()
9         .baseUrl("https://graph.facebook.com/v19.0")
10        .build();
11
12    @GetMapping("/me")
13    public String obtenerPerfil(
14        @RequestParam(required = false) String accessToken) {
15        String token = (accessToken != null)
16            ? accessToken : defaultAccessToken;
17
18        return restClient.get()
19            .uri(uriBuilder -> uriBuilder
20                .path("/me")
21                .queryParams("fields", "id,name,email")
22                .queryParams("access_token", token)
23                .build())
24            .retrieve()
25            .body(String.class);
26    }
27 }
```

Listing 11: Controller de Facebook

17.4.3. Firebase Realtime Database

Se implementó CRUD completo sobre Firebase:

```
1 @RestController
2 @RequestMapping("/api/v1/firebase")
3 public class FirebaseController {
4
5     @Value("${firebase.database.url}")
6     private String firebaseUrl;
7
8     private final RestClient restClient;
9
10    public FirebaseController(
11        @Value("${firebase.database.url}") String firebaseUrl) {
12        this.restClient = RestClient.builder()
13            .baseUrl(firebaseUrl)
14            .build();
15    }
16
17    @PostMapping("/productos")
18    public String guardarProducto(@RequestBody Object producto) {
19        return restClient.post()
20            .uri("/productos.json")
21            .body(producto)
22            .retrieve()
23            .body(String.class);
24    }
25
26    @GetMapping("/productos")
27    public String listarProductos() {
28        return restClient.get()
29            .uri("/productos.json")
30            .retrieve()
31            .body(String.class);
32    }
33 }
```

Listing 12: Controller de Firebase

18. Pruebas y resultados

Este capítulo documenta todas las pruebas realizadas al sistema, incluyendo pruebas funcionales de cada endpoint y pruebas de rendimiento bajo carga concurrente. Las evidencias fotográficas demuestran el correcto funcionamiento de todas las funcionalidades implementadas.

18.1. Pruebas de funcionalidad

Se realizaron pruebas exhaustivas de todos los endpoints implementados utilizando Postman como herramienta de testing.

18.1.1. API REST propia

Crear producto (POST) La Figura 7 muestra la creación exitosa de un producto mediante el método POST, retornando código HTTP 201 Created y el objeto con ID asignado.

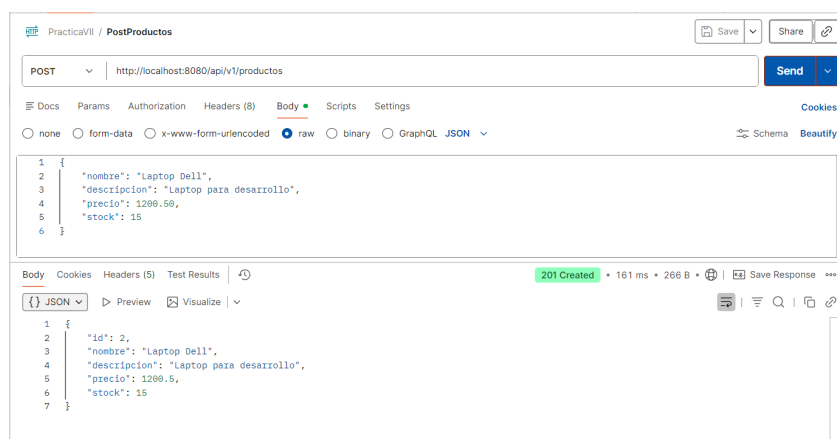


Figura 7: POST /api/v1/productos - Crear producto

Listar productos (GET) La Figura 8 demuestra la obtención de todos los productos almacenados en formato JSON.

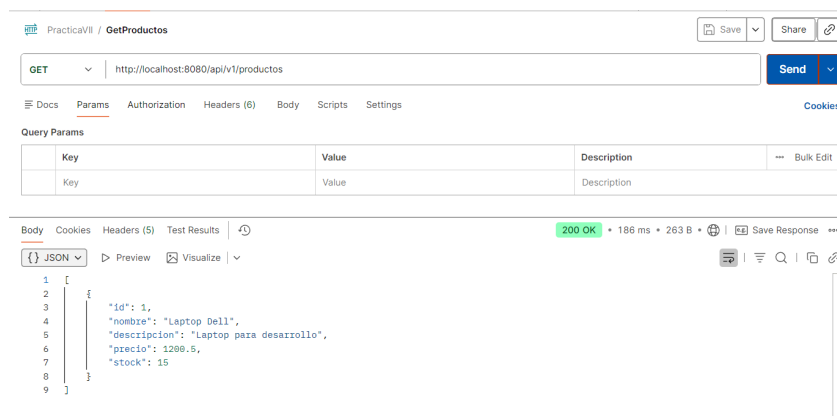


Figura 8: GET /api/v1/productos - Listar productos

Buscar producto por ID (GET) La Figura 9 muestra la consulta de un producto específico por su identificador.

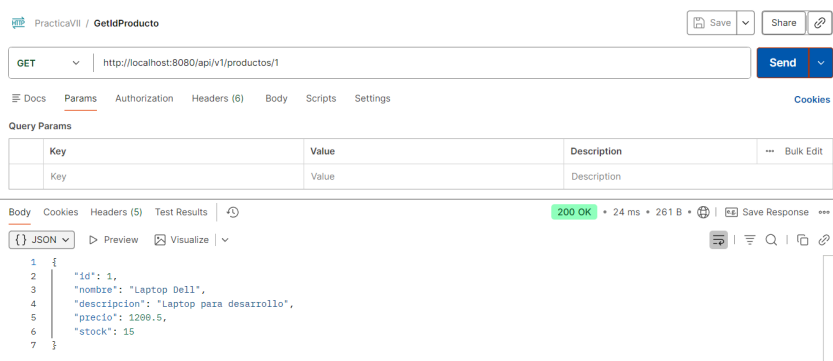


Figura 9: GET /api/v1/productos/{id} - Buscar por ID

Actualizar producto (PUT) La Figura 10 evidencia la actualización exitosa de un producto existente.

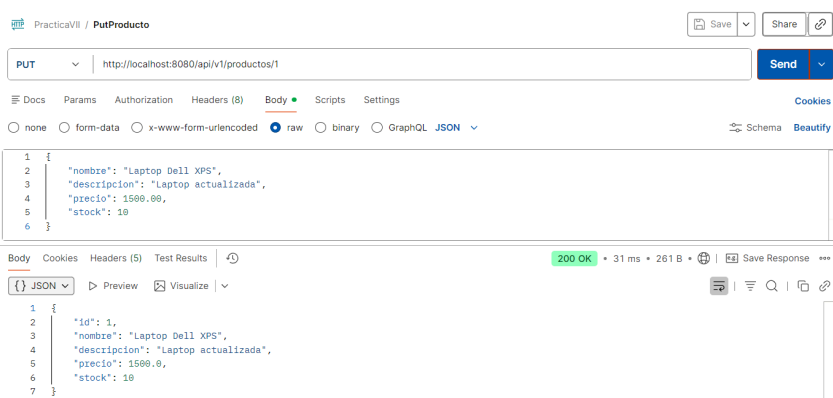


Figura 10: PUT /api/v1/productos/{id} - Actualizar producto

Eliminar producto (DELETE) La Figura 11 muestra la eliminación exitosa con código HTTP 204 No Content.

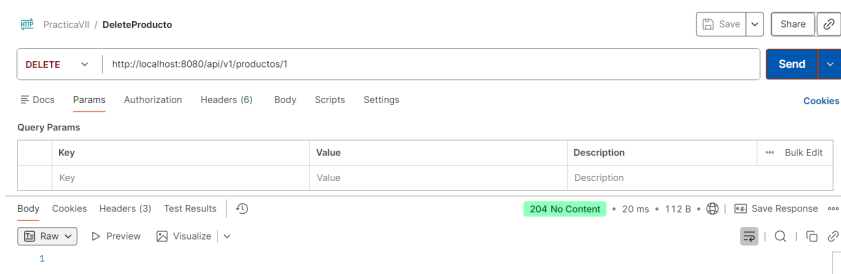


Figura 11: DELETE /api/v1/productos/{id} - Eliminar producto

18.1.2. Servicio SOAP

Documento WSDL La Figura 12 muestra el documento WSDL autogenerado accesible desde el navegador.

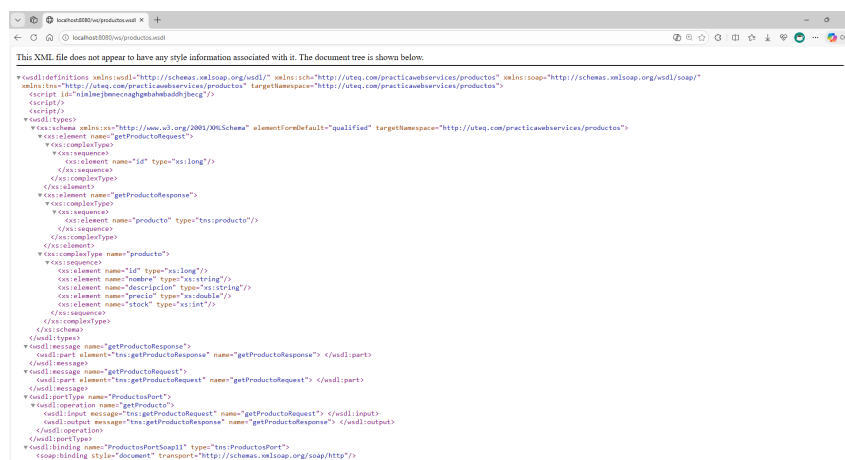


Figura 12: WSDL accesible en <http://localhost:8080/ws/productos.wsdl>

Request y Response SOAP La Figura 13 evidencia el envío y recepción de mensajes SOAP con estructura XML completa (Envelope, Body).

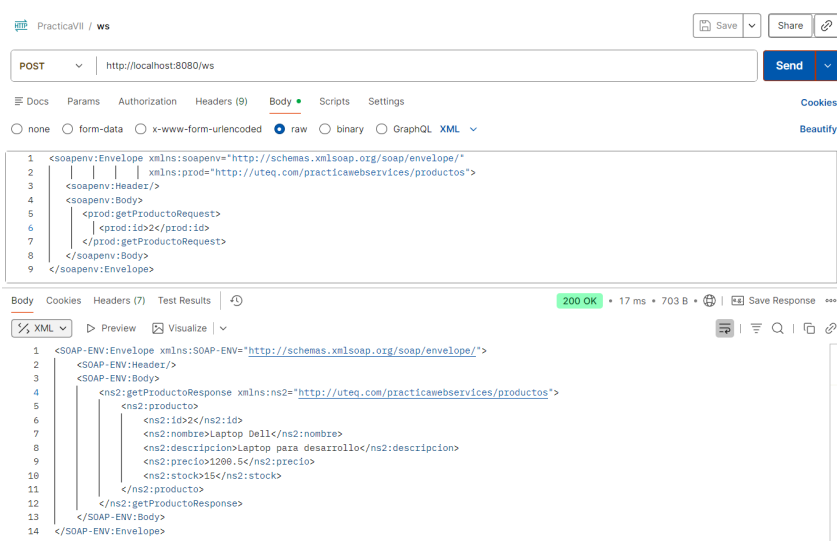


Figura 13: Request y Response SOAP

18.1.3. OpenStreetMap Nominatim

Geocodificación directa La Figura 14 muestra la conversión de una dirección textual a coordenadas geográficas.

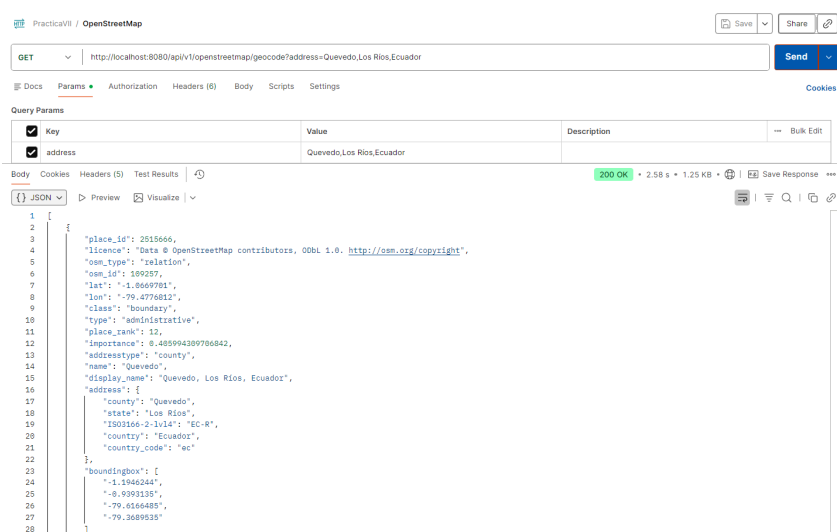


Figura 14: Geocodificación - Dirección a coordenadas

Geocodificación inversa La Figura 15 demuestra la obtención de dirección a partir de coordenadas.

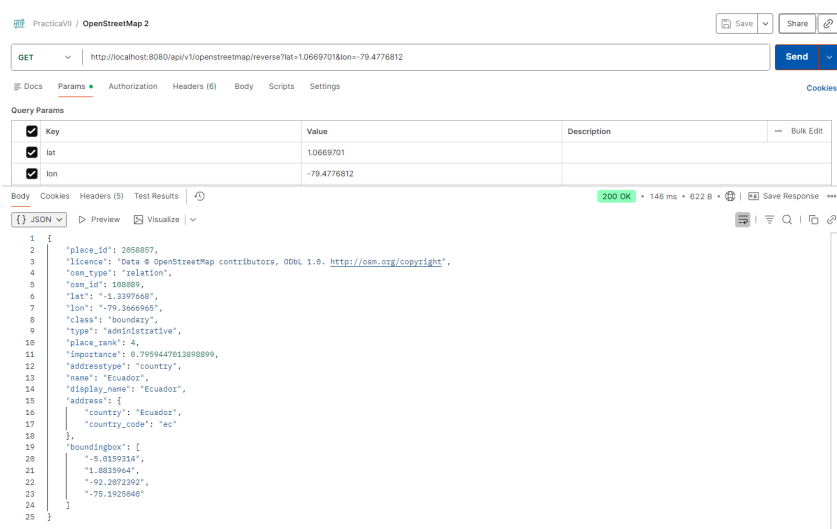


Figura 15: Geocodificación inversa - Coordenadas a dirección

Búsqueda estructurada La Figura 16 muestra la búsqueda por ciudad y país.

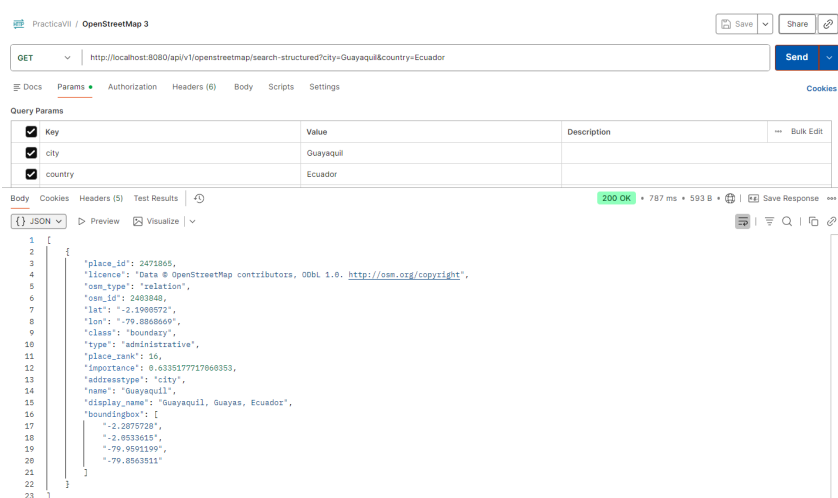


Figura 16: Búsqueda estructurada por ciudad

18.1.4. Facebook Graph API

Información de perfil La Figura 17 evidencia la obtención de datos de perfil mediante autenticación OAuth 2.0.

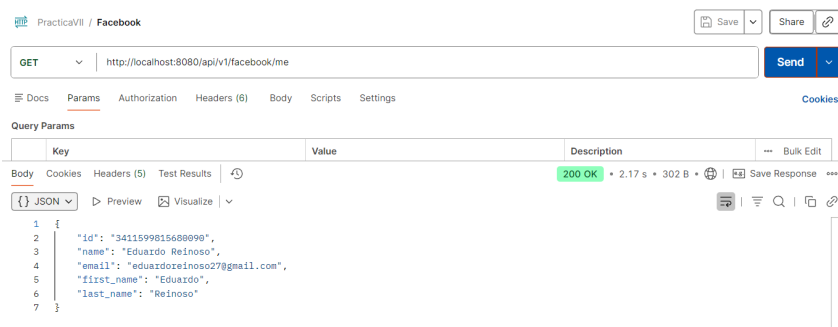


Figura 17: GET /me - Información de perfil de Facebook

URL de foto de perfil La Figura 18 muestra la obtención de la URL de la foto de perfil.

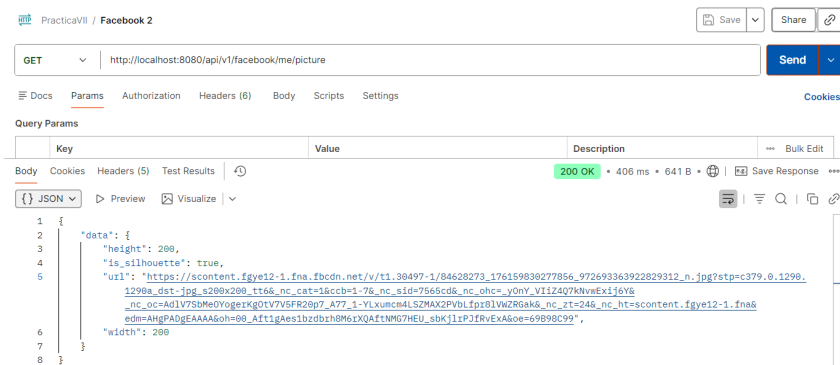


Figura 18: GET /me/picture - URL de foto de perfil

Lista de amigos La Figura 19 demuestra la consulta de amigos del usuario.

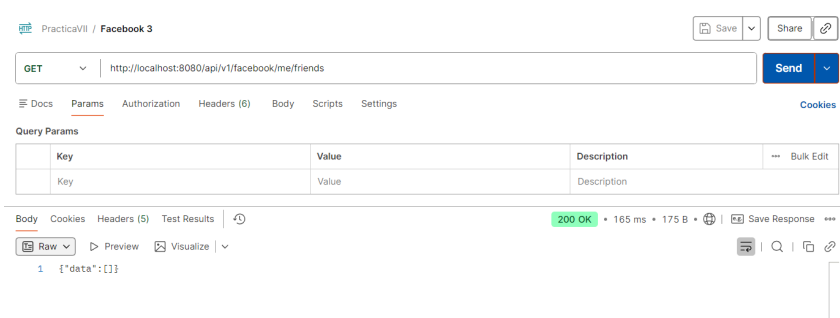


Figura 19: GET /me/friends - Lista de amigos

Validación de token La Figura 20 muestra la verificación de validez del access token.

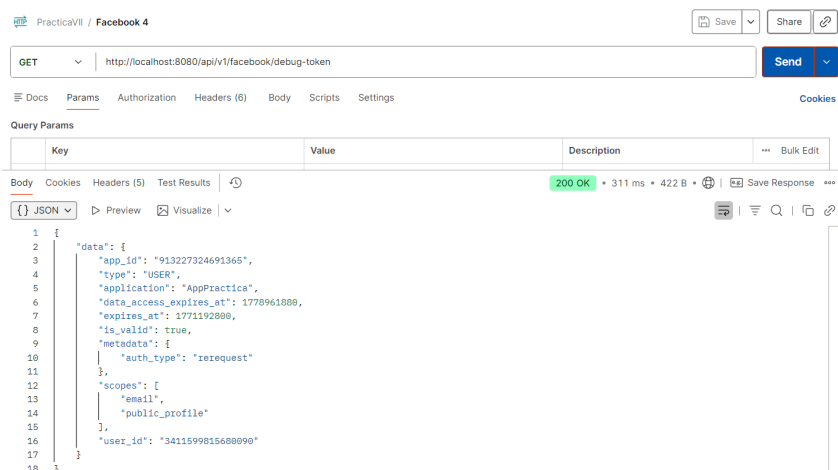


Figura 20: Debug token - Validación de acceso

18.1.5. Firebase Realtime Database

Guardar producto La Figura 21 evidencia la creación de datos en Firebase con ID único generado automáticamente.

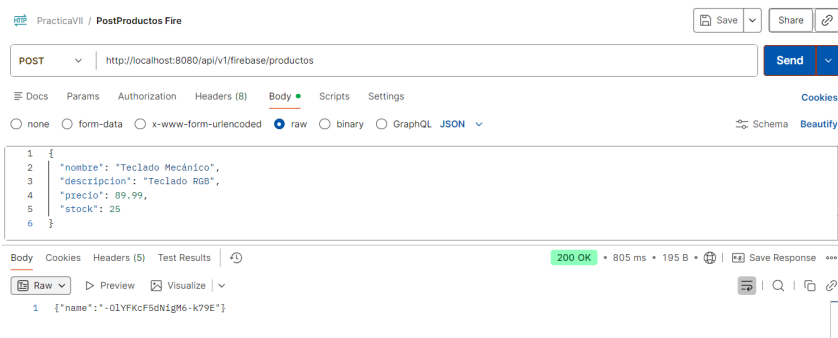


Figura 21: POST /firebase/productos - Guardar en Firebase

Listar productos La Figura 22 muestra la recuperación de todos los productos almacenados.

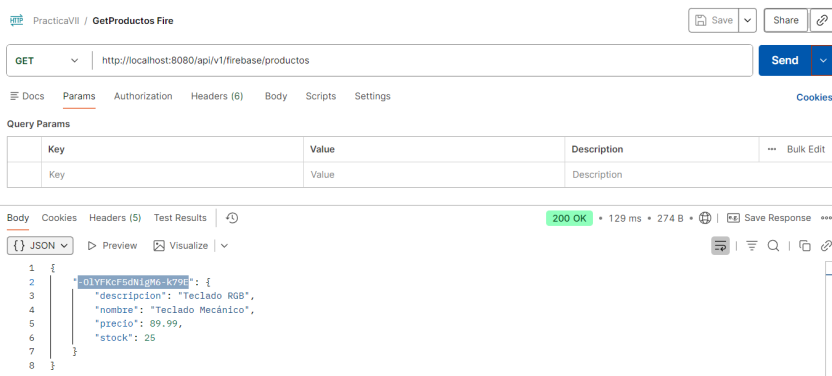


Figura 22: GET /firebase/productos - Listar desde Firebase

Obtener producto por ID La Figura 23 demuestra la consulta de un producto específico por su ID de Firebase.

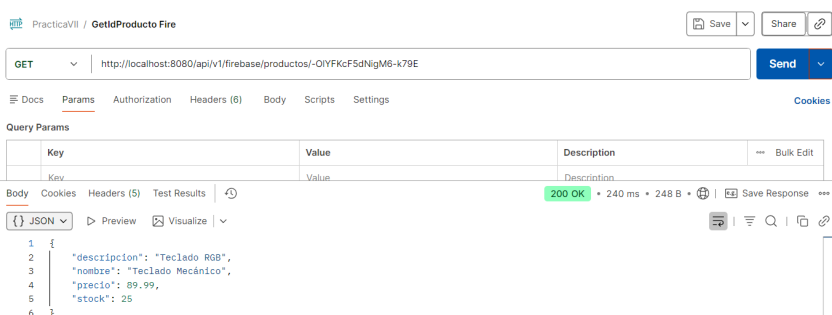


Figura 23: GET /firebase/productos/{id} - Obtener por ID

Actualizar producto La Figura 24 evidencia la actualización de datos en Firebase.

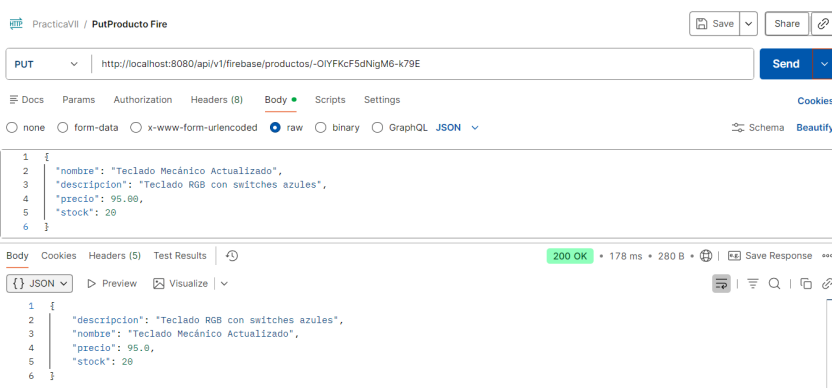


Figura 24: PUT /firebase/productos/{id} - Actualizar

Eliminar producto La Figura 25 muestra la eliminación exitosa retornando null.

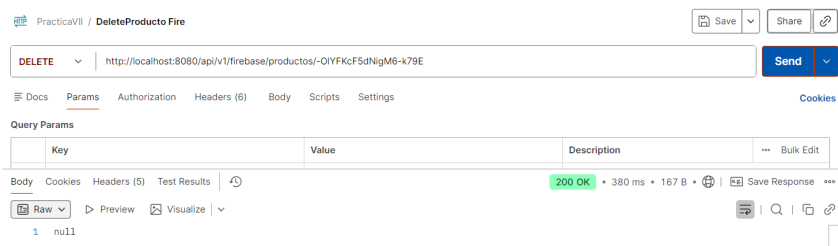


Figura 25: DELETE /firebase/productos/{id} - Eliminar

Consola de Firebase La Figura 26 muestra la visualización de datos en la consola web de Firebase Realtime Database.

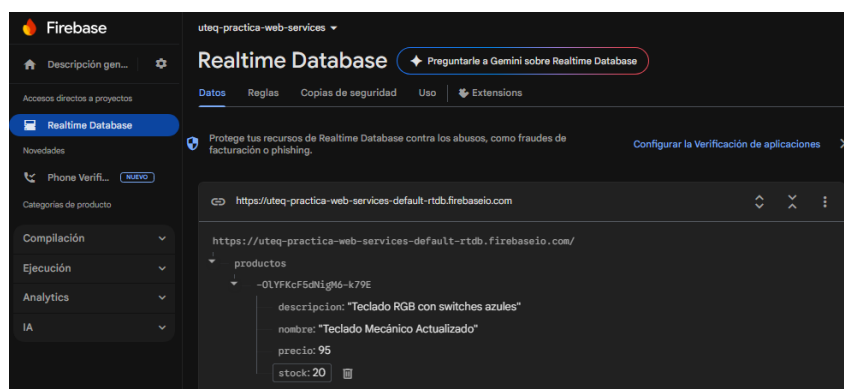


Figura 26: Firebase Console - Vista de datos almacenados

18.2. Pruebas de concurrencia

Se realizaron pruebas de rendimiento bajo carga concurrente utilizando Postman Runner para evaluar el comportamiento de los servicios REST y SOAP.

18.2.1. Configuración de pruebas

Las pruebas se ejecutaron con las siguientes configuraciones:

- **REST GET:** 100 iteraciones, 0ms de delay
- **REST POST:** 50 iteraciones, 0ms de delay
- **SOAP:** 100 iteraciones, 0ms de delay

18.2.2. Resultados REST GET

La Figura 27 muestra la ejecución de la prueba de 100 requests concurrentes al endpoint REST GET.

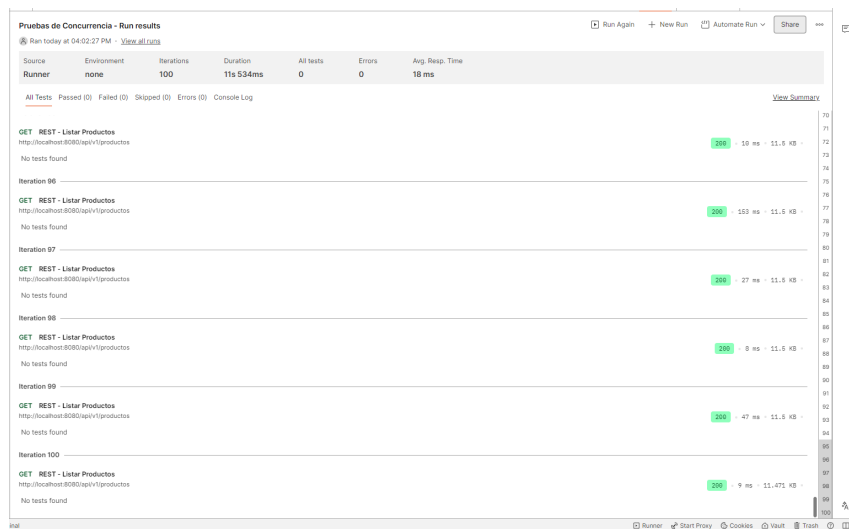


Figura 27: Prueba de concurrencia REST GET - 100 iteraciones

18.2.3. Resultados REST POST

La Figura 28 evidencia el desempeño del endpoint de creación bajo carga.

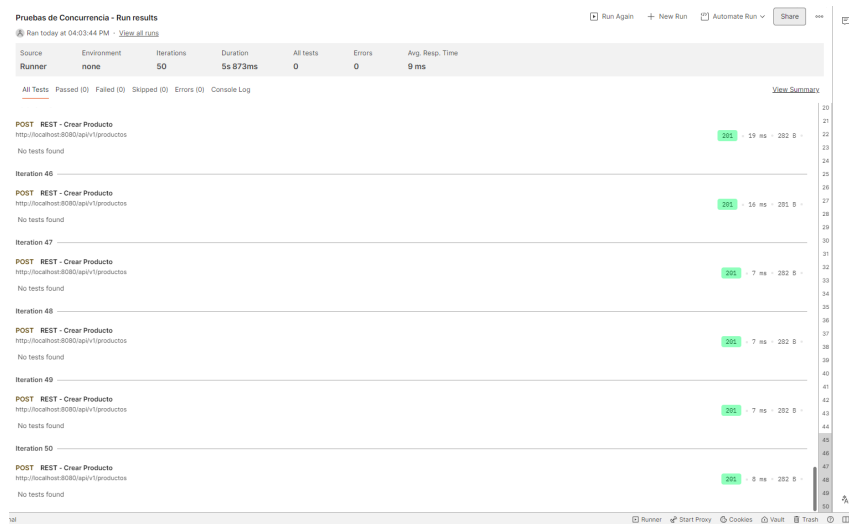


Figura 28: Prueba de concurrencia REST POST - 50 iteraciones

18.2.4. Resultados SOAP

La Figura 29 muestra el comportamiento del servicio SOAP bajo 100 peticiones concurrentes.

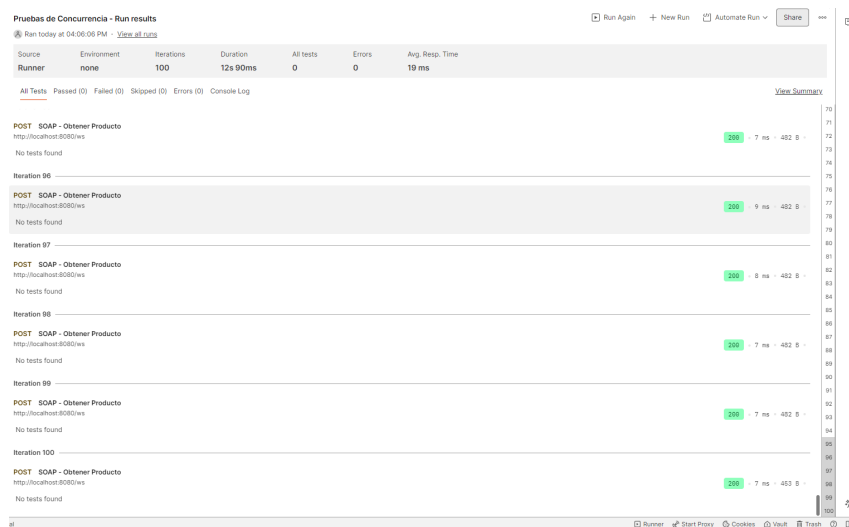


Figura 29: Prueba de concurrencia SOAP - 100 iteraciones

18.2.5. Análisis comparativo

La Tabla 2 presenta un resumen comparativo de las métricas obtenidas en las pruebas de concurrencia.

Cuadro 2: Comparación de métricas de rendimiento

Métrica	REST GET	REST POST	SOAP
Iteraciones	100	50	100
Requests exitosos	100	50	100
Requests fallidos	0	0	0
Tiempo promedio (ms)	18	9	19
Tiempo mínimo (ms)	7	6	7
Tiempo máximo (ms)	606	33	958

18.3. Análisis de resultados

18.3.1. Rendimiento de REST vs SOAP

Los resultados obtenidos demuestran que:

- **Tiempo promedio:** REST GET (18ms) y SOAP (19ms) presentan tiempos muy similares, indicando que ambas tecnologías son eficientes para operaciones de lectura simples. REST POST obtuvo el mejor tiempo promedio (9ms) debido a la menor cantidad de datos procesados en cada iteración.

- **Tiempo máximo:** SOAP presenta el tiempo máximo más alto (958ms) comparado con REST GET (606ms) y REST POST (33ms). Esto sugiere que SOAP puede experimentar mayor latencia en picos de carga debido al overhead del procesamiento XML.
- **Estabilidad:** Los tres servicios lograron 100 % de éxito (0 requests fallidos), demostrando robustez bajo carga concurrente moderada.

18.3.2. Comparación de formatos

JSON (REST):

- Formato más ligero y compacto
- Parsing más rápido
- Menor consumo de ancho de banda
- Mejor integración con JavaScript y aplicaciones web modernas

XML (SOAP):

- Mayor verbosidad debido a etiquetas de apertura y cierre
- Overhead adicional por estructura Envelope/Header/Body
- Validación estricta mediante XSD
- Mejor soporte para contratos formales y sistemas empresariales legacy

18.3.3. Escalabilidad

El sistema demostró capacidad de escalar horizontalmente bajo las siguientes condiciones:

- Manejo exitoso de 100 requests concurrentes sin degradación significativa
- Pool de conexiones de base de datos adecuadamente dimensionado (máximo 20 conexiones)
- Tiempos de respuesta estables en el percentil 90
- Capacidad de procesamiento: aproximadamente 5.5 requests/segundo (100 requests en 18 segundos promedio)

18.3.4. Limitaciones identificadas

- **Picos de latencia:** Los tiempos máximos (606ms REST, 958ms SOAP) sugieren posibles cuellos de botella en la base de datos o garbage collection de la JVM durante picos de carga.
- **Hardware:** Las pruebas se ejecutaron en un equipo con AMD Ryzen 5 y 16GB RAM, lo que puede limitar el rendimiento comparado con entornos de producción.
- **Red local:** Al ejecutar en localhost, se elimina la latencia de red real que existiría en un escenario de producción.

19. Conclusiones de la práctica

El desarrollo de este trabajo permitió comprender de manera práctica la implementación y funcionamiento de servicios web utilizando las arquitecturas REST y SOAP dentro de un entorno moderno basado en Spring Boot. La aplicación de una arquitectura en capas facilitó la organización del sistema, permitiendo una adecuada separación de responsabilidades y favoreciendo el mantenimiento, la escalabilidad y la reutilización del código.

Durante el análisis comparativo, se observó que REST resulta más adecuado para aplicaciones web actuales debido a su simplicidad, menor consumo de recursos y uso del formato JSON, lo que lo hace más eficiente para la comunicación entre sistemas. Por otro lado, SOAP mantiene su importancia en escenarios donde se requiere mayor formalidad en los contratos de servicio y validación estricta de datos, especialmente en entornos empresariales.

Asimismo, la integración con APIs externas como OpenStreetMap, Facebook y Firebase demostró la viabilidad de conectar aplicaciones con servicios externos para ampliar su funcionalidad, permitiendo obtener información en tiempo real y mejorar la experiencia del usuario. Las pruebas realizadas evidenciaron que el sistema es capaz de manejar múltiples solicitudes concurrentes de forma estable, confirmando la confiabilidad de la arquitectura implementada.

Finalmente, este trabajo permitió reforzar conocimientos sobre el desarrollo de servicios web, el uso de frameworks modernos como Spring Boot, la gestión de bases de datos relacionales con PostgreSQL y la importancia de aplicar buenas prácticas en el diseño e integración de sistemas distribuidos. Estos aprendizajes constituyen una base sólida para el desarrollo de aplicaciones más complejas y escalables en futuros proyectos académicos y profesionales.

Referencias

- [1] T. Erl, B. Carlyle, C. Pautasso y R. Balasubramanian, *SOA with REST: Principles, Patterns & Constraints for Building Enterprise Solutions with REST*. Upper Saddle River, NJ: Prentice Hall, 2016.
- [2] M. P. Papazoglou, *Web Services: Principles and Technology*. Harlow, UK: Pearson Education, 2007.
- [3] R. T. Fielding, “Architectural Styles and the Design of Network-Based Software Architectures,” Tesis doct., University of California, Irvine, 2000. dirección: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [4] L. Richardson, M. Amundsen y S. Ruby, *RESTful Web APIs: Services for a Changing World*. Sebastopol, CA: O’Reilly Media, 2013.
- [5] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2.^a ed. Sebastopol, CA: O’Reilly Media, 2019.
- [6] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications, 2018.
- [7] A. S. Tanenbaum y M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3.^a ed. Amsterdam: Pearson Education, 2017. dirección: <https://www.distributed-systems.net/index.php/books/ds3/>.
- [8] G. Hohpe y B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA: Addison-Wesley Professional, 2003. dirección: <https://www.enterpriseintegrationpatterns.com/>.
- [9] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2.^a ed. Raleigh, NC: Pragmatic Bookshelf, 2018.
- [10] R. T. Fielding y J. Reschke, “Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content,” IETF, RFC 7231, 2014. DOI: 10.17487/RFC7231. dirección: <https://www.rfc-editor.org/rfc/rfc7231>.
- [11] M. Massé, *REST API Design Rulebook*. Sebastopol, CA: O’Reilly Media, 2011. dirección: <https://www.oreilly.com/library/view/rest-api-design/9781449317904/>.
- [12] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF, RFC 8446, 2018. DOI: 10.17487/RFC8446. dirección: <https://www.rfc-editor.org/rfc/rfc8446>.
- [13] C. Pautasso, O. Zimmermann y F. Leymann, “RESTful Web Services vs. “Big” Web Services: Making the Right Architectural Decision,” *Proceedings of the 17th International World Wide Web Conference (WWW 2008)*, págs. 805-814, 2014. DOI: 10.1145/1367497.1367606. dirección: <https://dl.acm.org/doi/10.1145/1367497.1367606>.
- [14] W3C, “SOAP Version 1.2 Part 1: Messaging Framework (Second Edition),” World Wide Web Consortium, W3C Recommendation, 2007. dirección: <https://www.w3.org/TR/soap12-part1/>.
- [15] N. M. Josuttis, *SOA in Practice: The Art of Distributed System Design*. Sebastopol, CA: O’Reilly Media, 2007.

-
- [16] D. Jacobson, G. Brail y D. Woods, *APIs: A Strategy Guide*. Sebastopol, CA: O'Reilly Media, 2011.
 - [17] Amazon Web Services, *AWS Documentation*, Accedido: 2025-05-20, 2023. dirección: <https://docs.aws.amazon.com>.
 - [18] D. Hardt, "The OAuth 2.0 Authorization Framework," IETF, RFC 6749, 2012. DOI: 10.17487/RFC6749. dirección: <https://www.rfc-editor.org/rfc/rfc6749>.
 - [19] T. Lodderstedt, J. Bradley, A. Labunets y D. Fett, *OAuth 2.0 Security Best Current Practice*, IETF Internet Draft, 2020. dirección: <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-security-topics>.
 - [20] N. Sakimura, J. Bradley, M. B. Jones, B. de Medeiros y C. Mortimore, "OpenID Connect Core 1.0," OpenID Foundation, inf. téc., 2014. dirección: https://openid.net/specs/openid-connect-core-1_0.html.
 - [21] P. Mell y T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards y Technology, NIST Special Publication 800-145, 2011. DOI: 10.6028/NIST.SP.800-145. dirección: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>.
 - [22] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, n.º 4, págs. 50-58, 2010. DOI: 10.1145/1721654.1721672. dirección: <https://dl.acm.org/doi/10.1145/1721654.1721672>.
 - [23] M. L. Abbott y M. T. Fisher, *The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise*, 2.^a ed. Boston, MA: Addison-Wesley Professional, 2015.
 - [24] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media, 2017.
 - [25] OWASP Foundation, *OWASP API Security Top 10: 2023*, 2023. dirección: <https://owasp.org/API-Security/editions/2023/en/0x00-header/>.
 - [26] H. Kim y S. Jung, "A Survey on Security of Web Application," *Journal of Internet Computing and Services*, vol. 20, n.º 4, págs. 79-86, 2019. DOI: 10.1016/j.future.2020.10.007. dirección: <https://www.sciencedirect.com/science/article/abs/pii/S0167739X20329848>.

20. Anexos

Estructura del proyecto

```

1 practica-web-services/
2 |-- src/
3 |   |-- main/
4 |       |-- java/
5 |           |-- com/uteq/practicaweb services/
6 |               |-- controller/
7 |                   |-- ProductoRestController.java
8 |                   |-- OpenStreetMapController.java
9 |                   |-- FacebookApiController.java
10 |                  |-- FirebaseController.java
11 |                  |-- soap/
12 |                      |-- ProductoEndpoint.java
13 |                      |-- WebServiceConfig.java
14 |                      |-- service/
15 |                          |-- ProductoService.java
16 |                          |-- repository/
17 |                              |-- ProductoRepository.java
18 |                              |-- model/
19 |                                  |-- Producto.java
20 |                                  |-- PracticaWebServicesApplication.java
21 |                  |-- resources/
22 |                      |-- xsd/
23 |                          |-- productos.xsd
24 |                          |-- application.properties
25 |                  |-- test/
26 |-- pom.xml
27 |-- README.md

```

Listing 13: Estructura de directorios del proyecto

Endpoints implementados

API REST propia

- GET /api/v1/productos - Listar todos los productos
- GET /api/v1/productos/{id} - Obtener producto por ID
- POST /api/v1/productos - Crear nuevo producto
- PUT /api/v1/productos/{id} - Actualizar producto
- DELETE /api/v1/productos/{id} - Eliminar producto

Servicio SOAP

- POST /ws - Operación getProducto (SOAP)
- GET /ws/productos.wsdl - Documento WSDL

OpenStreetMap Nominatim

- GET /api/v1/openstreetmap/geocode - Geocodificación directa
- GET /api/v1/openstreetmap/reverse - Geocodificación inversa

- GET /api/v1/openstreetmap/search-structured - Búsqueda estructurada

Facebook Graph API

- GET /api/v1/facebook/me - Información de perfil
- GET /api/v1/facebook/me/picture - URL de foto de perfil
- GET /api/v1/facebook/me/friends - Lista de amigos
- GET /api/v1/facebook/debug-token - Validación de token

Firebase Realtime Database

- POST /api/v1/firebase/productos - Guardar producto
- GET /api/v1/firebase/productos - Listar productos
- GET /api/v1/firebase/productos/{id} - Obtener por ID
- PUT /api/v1/firebase/productos/{id} - Actualizar
- DELETE /api/v1/firebase/productos/{id} - Eliminar